Big Data Management — P1 Report

Cymatics: Sound-Driven Pattern Generation & Analysis Pipeline

Arman Bazarchi
Brisa Fernanda Cisneros Cervantes

Contents

1 Instructions

Repository access: All source code and files are available on GitHub:
`https://github.com/armanbzi/BDM-Cymatics.git`

1.1 Docker Compose

We provide a `docker-compose.yml` file to run the local infrastructure required for our data ingestion paths.

What it runs

- **Zookeeper** (`confluentinc/cp-zookeeper:7.5.0`) on port `2181`
- **Kafka** (`confluentinc/cp-kafka:7.5.0`) on port `9092`
  - `KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092`
  - `KAFKA_AUTO_CREATE_TOPICS_ENABLE=true`
- **Kafka UI** (`provectuslabs/kafka-ui:latest`) on port `8085` (Web UI for Kafka topic/message inspection)
  - Kafka uses dual listeners to support both inter-container communication ('INTERNAL://kafka:29092') and host-machine access ('EXTERNAL://localhost:9092'), to ensure availability for both UI and the hot-path
- **MinIO** (`minio/minio:latest`) on ports `9000` (API) and `9001` (console)
  - Default credentials are defined via environment variables in the compose file
- **PostgreSQL** (`postgres:16-alpine`) on port `5432` (Airflow metadata database)
- **Airflow Init** (`Custom via Dockerfile to fix issue with ffmpeg availability`) (One-shot DB migration + admin user creation)
  - The custom Dockerfile extends 'apache/airflow:2.9.3-python3.11' by installing ffmpeg (system-level, for audio decoding) and all Python dependencies ('python-dotenv', 'requests', 'numpy', 'scipy', 'minio', 'pyarrow') at build time.
- **Airflow Webserver** (`Custom via Dockerfile to fix issue with ffmpeg availability`) on port `8080` (Web UI for DAG management)
- **Airflow Scheduler** (`Custom via Dockerfile to fix issue with ffmpeg availability`) on port (Executes scheduled DAGs)
- **SonarQube** (`sonarqube:lts-community`) on host port `9090` (container listens on `9000`); web UI and REST API at `http://localhost:9090`
  - Uses a dedicated PostgreSQL service (`sonar-postgres`, `postgres:16-alpine`) for SonarQube's application database (not exposed on the host by default).
  - JDBC credentials are injected from `.env` via `SONAR_JDBC_USERNAME` / `SONAR_JDBC_PASSWORD` (must match `POSTGRES_USER` / `POSTGRES_PASSWORD` on `sonar-postgres`).

  - `SONAR_FORCEAUTHENTICATION=false` simplifies local development; tighten for production deployments.

Volumes (persistence)

- `./kafka` → Kafka data directory, we store kafka files in the project's working directory in the folder"kafka"
- `./minio` → MinIO object storage data, we store our files in the project's working directory in the folder"minio"
- `./airflow-pgdata` → PostgreSQL data, stores Airflow's internal state.
- `./airflow-logs` → For storing Airflow task logs.
- Named volumes `sonarqube-data`, `sonarqube-extensions`, `sonarqube-logs` → SonarQube application data, plugins/extensions, and logs
- Named volume `sonar-pgdata` → PostgreSQL data for the SonarQube database

When to run it

Run the infrastructure before executing any scripts or notebooks(unless services are already running):

```
docker compose up -d
```

1.2 Environment Variables

See `env.example` for default values. The most relevant variables are:

MinIO

- `MINIO_ENDPOINT` (default: `localhost:9000`)
- `MINIO_ACCESS_KEY` (default: `admin`)
- `MINIO_SECRET_KEY` (default: `password`)
- `MINIO_SECURE` (set to `false` for local Docker setup)
- `MINIO_BUCKET` (default: `landing-zone`)

Kafka

- `KAFKA_BOOTSTRAP_SERVERS` (default: `localhost:9092`)
- `KAFKA_TOPIC` (topic for storing event messages(warm)
- `KAFKA_TOPIC_HOT` (hot-path topic, default: `landing-zone-events-hot-path`)

External sources

- `FREESOUND_API_KEY` required for cold-path Freesound ingestion
  Get one at Freesound API token page
- `ESC50_BASE_PATH` (optional local path, the script auto-downloads if missing)

Airflow

- `AIRFLOW_USERNAME` (default: 'admin')
- `AIRFLOW_PASSWORD` (default: 'admin')

SonarQube

- `SONAR_JDBC_USERNAME` / `SONAR_JDBC_PASSWORD` — PostgreSQL credentials for the **Sonar-only** database used by `sonar-postgres` and SonarQube JDBC (defaults in compose: `sonar` / `sonar`). Must stay consistent between the DB service and SonarQube.
- `SONAR_TOKEN` (optional, preferred) — user token for `sonar-scanner` and Sonar REST API calls from `orchestrate.py`;must be created and used for running code quality checks via SonarQube, create under My Account → Security.
- `SONAR_USERNAME` / `SONAR_PASSWORD` (optional) — SonarQube **web UI** login used when no token is set (manual orchestrator option 7).
- `SONAR_LOGIN` — optional alias for `SONAR_USERNAME`.

Note: `SONAR_JDBC_*` variables are *not* the Sonar web password; they only configure the Sonar PostgreSQL role.

1.3 SonarQube (static code analysis)

SonarQube (`sonarqube:lts-community`) provides code quality checks of our project: bugs, vulnerabilities, code smells, duplication, coverage (if tests exist), and maintainability ratings. The project is configured via `sonar-project.properties` (project key `bdm-cymatics`, Python 3.11, exclusions for large local folders such as `data/`, `minio/`, `kafka/`).

How we run it locally:

1. Start Docker services: `docker compose up -d` (SonarQube becomes available at `http://localhost:9090` after the health check stabilises; first boot can take about 90 seconds), then login(default usename and password are both admin) and generate a token and add it to the .env.
2. Install the SonarScanner CLI on the host (e.g. `brew install sonar-scanner`).
3. After providing the token in .env file, we are able to run code quality checks via our orchestrator and view the results there or in the dashboard.

1.4 Run Guide

Manual Orchestrator (recommended)

```
docker compose up -d
python orchestrate.py
```

It displays an interactive menu to run any pipeline flow with live CPU/RAM/disk monitoring:

#	Flow	Mode
1	Warm path	Near-real-time
2	Hot path (producer + consumer)	Real-time (parallel processes)
3	Cold path: Freesound	Batch (API)
4	Cold path: ESC-50	Batch (dataset)
5	Trusted zone processing	Batch enrichment
6	Sync Delta Lake	On-demand sync
7	SonarQube code analysis	Static analysis (`sonar-scanner` + dashboard)

Warm Path

```
docker compose up -d
python landing_zone/warm_path/landing_zone_warm.py
```

Hot Path

Run in two separate terminals:

```
docker compose up -d
python landing_zone/hot_path/landing_zone_hot_consumer.py

python landing_zone/hot_path/landing_zone_hot_producer.py
```

Cold path — Freesound

```
docker compose up -d
python landing_zone/cold_path/cold_freesound.py 50
```

Cold path — ESC-50

```
docker compose up -d
python landing_zone/cold_path/cold_esc50.py 50
```

Airflow (automated scheduling)

Navigate to `http://localhost:8080`, log in with `AIRFLOW_USERNAME`/`AIRFLOW_PASSWORD`, and unpause `cold_freesound_ingestion`.

The DAG runs weekly, ingesting up to 250 new sounds with incremental checkpointing. This means the system stores the last ingestion state in MinIO (in a bucket), allowing subsequent runs to retrieve only records added after the previous execution date.

2 Contextualization

2.1 Domain and Problem Statement

This project addresses the **Acoustics & Cymatics** domain, specifically focusing on transforming audio signals into visual cymatics patterns — the geometric shapes that emerge when sound vibrates a physical medium such as water or a metal plate (via sand).

> *"Given an audio signal (recorded live or sourced from datasets/APIs), generate a visual representation of its cymatics pattern (image and video showing its evolution), extract acoustic features, and organize all artifacts into a structured, queryable dataset for analysis."*

Cymatics bridges physics, music, and visual art. While the phenomenon has been studied since Ernst Chladni's 18th-century experiments, there is no large-scale, structured dataset that pairs audio recordings with their corresponding cymatics visualizations and acoustic metadata. This project aims to fill that gap by building an automated pipeline that ingests audio, generates cymatics artifacts, and stores everything in a well-organized data lakehouse.

2.2 Business Value

The pipeline delivers value across multiple domains:

- **Scientific Research:** Researchers in acoustics and physics can study frequency–pattern relationships at scale, comparing how different frequencies, amplitudes, and waveforms produce distinct geometric structures.
- **Music & Sound Design:** Audio engineers and musicians can visualize the "shape" of their sounds, enabling a new modality for understanding and comparing audio content.
- **Art & Creative Applications:** Cymatics patterns are visually striking. The pipeline can generate high-resolution artwork from any audio source, with potential applications in generative art, NFTs, and visual installations.
- **Education:** Interactive cymatics visualizations make abstract concepts like wave interference, resonance, and standing waves tangible for students.
- **Pattern Classification:** The generated dataset enables training ML models to classify sounds or visual patterns — essentially "seeing" sound.

2.3 Data Sources and Categories

The pipeline processes structured, semi-structured, and unstructured data.

Table 1: Data Sources and Categories

Category	Data	Format	Source
Structured	Audio metadata columns (frequency, amplitude, RMS, duration, file paths, ...)	CSV / Parquet	Generated by the pipeline/Expanding source dataset (if any)
Semi-structured	Per-observation metadata (peak frequencies, spectral peaks, ...)	JSON	Generated by Kafka
Semi-structured	Streaming Event Messages when using Kafka in warm/hot paths	JSON	Generated by Kafka
Unstructured	Raw audio recordings	.wav	Live mic recording, FreeSound API, audio datasets, ...
Unstructured	Cymatics images (peak frequency pattern)	.png	Generated by the pipeline
Unstructured	Cymatics videos (patterns appearing)	.mp4	Generated by the pipeline

Data Sources

1. **Live Microphone Recording (Warm Path)** — the user records audio directly, and images and videos are generated and stored along with their metadata. This represents a warm/near-real-time ingestion path.
2. **Live Visualization (Hot Path)** — user is also able to visualize and see the patterns appearing and changing in real time via live microphone recording and store batches of audio observations and corresponding metadata as the microphone records (Hot Path).
3. **FreeSound API (Cold Path)** — programmatic access to a large repository of Creative Commons audio files, enabling batch ingestion of diverse sound samples (musical instruments, environmental sounds, speech, etc.), Automated via Airflow (weekly, up to 250 new sounds).
4. **ESC-50 Dataset (Cold Path)** — Batch ingestion of a pre-existing environmental sound dataset (50 categories). Auto-downloaded if not present in working directory, filtered to our desired categories (rain, sea waves, thunder, crickets, birds, etc.).

One observation in the final dataset consists of:

- Raw audio file (.wav)
- Generated cymatics image (.png)
- Generated cymatics video (.mp4)
- Streaming event messages that appeared if Kafka was used (usually JSON).
- Textual columns providing audio/visualization features, file paths (CSV, Parquet).

2.4 Cymatics Generation Engine

We implement a wave-source interference method with a **two-zone plate** and **harmonic-aware peak selection**:

1. **Two concentric zones**: an **inner disk** and an **outer ring**, each with its own wave sources and spatial scaling, to represent the pattern difference that appears via same audio vibrations and on same surface but with different water depth levels.
2. **Wave sources** are placed around zone boundaries, each source emits waves based on detected spectral frequencies. Alternating sine and cosine functions are applied between sources.
3. **Interference rendering**: the vector sum of displacements is computed on a grid and converted into a "water surface" visualization.
4. **Nodal Lines Detection:** points where displacement ≈ 0, these are where sand/particles would accumulate in a real Chladni plate, forming geometric patterns.
5. **Peak frequency**(per observation): rather than taking the single strongest FFT bin (which can be a harmonic), we pick the frequency that **appears most over time** across overlapping windows and fold harmonics down to the fundamental (energy-weighted).
6. **Rendering:** Two-color visualization with zero-crossing lines for geometric detail, images are 2048×2048; videos are 600×600 at 30 fps.

2.4.1 Peak Detection (Harmonic-Aware algorithm)

For each 5-second observation, we select the "dominant" frequency as the one that **appears most over time**, while being robust to harmonics.

High-level approach

- Split the audio into overlapping windows (0.25s, 50% hop).
- For each window, compute the FFT and extract the dominant peak (`freq_bin`).
- Build a harmonic-aware histogram by boosting **divisors** of each detected peak (e.g., 120 contributes to 60).
- Select the fundamental frequency with the strongest energy-weighted support.
- Choose the "best" window among those where the detected peak corresponds to the fundamental or its harmonics.
- Generate the cymatics image and video labels based on this selected peak.

3 Implementation

3.1 Landing-Zone

The landing zone ingests and stores **raw audio and basic metadata only**. No cymatics generation or spectral feature extraction happens here — that is deferred to the trusted zone. All data is stored in MinIO (bucket: 'landing-zone'), Kafka carries event messages only (references/paths), not audio files.

Metadata schema (landing zone): 'uuid', 'source_id', 'time_recorded/added', 'duration', 'audio_size', 'audio_path', 'audio_format', 'source', 'peak_frequency_hz'. Cold paths additionally store: 'tags', 'description', 'category'.

Metadata formats: Every ingestion path writes to both 'metadata/observations.csv' and 'metadata/observations.parquet' simultaneously.

Paths used in MinIO(landing-zone):

- `audio/warm-path/<peak_freq>/<uuid>-<peak_freq>.wav`
- `audio/hot-path/raw/<uuid>.wav`(temporary, deleted by consumer)
- `audio/hot-path/<peak_freq>/<uuid>-<peak_freq>.wav`(persistent, after ingested completely by consumer)
- `audio/Freesound/<peak_freq>/<cat>_<freq>_<uuid>.wav`
- `audio/ESC-50/<peak_freq>/<cat>_<freq>_<uuid>.wav`

- Metadata files:

- `landing-zone/metadata/observations.csv`
- `landing-zone/metadata/observations.parquet`
- `landing-zone/metadata/freesound_last_ingestion.txt`(for auto Airflow ingestion, to only store new added records)

Notes

- The consumer in hot path copies raw audio into the correct folder and removes the raw object, if server-side copy fails, it re-uploads and then deletes raw.

3.1.1 Ingestion paths

Cold Path — Freesound API (batch ingestion) This path focuses on **batch ingestion** from external audio sources (APIs or datasets like FreeSound), using authentication tokens/credentials instead of live microphone input.

Freesound API:

1. Connects to the Freesound API v2 using an API key(API token must be provided as a variable in .env file).
2. Searches across 15 randomized query categories that we found suitable for Cymatics:
   - rain, church bells, crickets, thunder, water drops, sea waves, wind, singing bowl, tuning fork, bird song, cat purring, dog bark, waterfall, ocean, forest
3. For each sound ($\leq$ 10 seconds):
   - Downloads the original audio and decodes it to mono WAV via `ffmpeg`.
   - Performs harmonic-aware peak detection.
   - Uploads the WAV file to `audio/Freesound/<peak_freq>/`.
   - Appends metadata with Freesound-specific fields (tags, description, category).
4. **Deduplication:** skips sounds whose `source_id` (Freesound ID) already exists in the metadata CSV.
5. **Incremental ingestion:** supports `created_after` date filtering and persists the last ingestion timestamp in MinIO (`metadata/freesound_last_ingestion.txt`), so Airflow-triggered runs only fetch newly created sounds after previous ingestion.

ESC-50 pre-existing dataset:

1. We first download the ESC-50 dataset from GitHub if not present locally.
2. We filter to 17 categories that can be interesting for Cymatics:
   - rain, sea_waves, thunderstorm, crickets, chirping_birds, water_drops, wind, church_bells, cat, dog, crow, rooster, hen, insects, frog, crackling_fire, pouring_water
3. We prioritize different categories in each batch for variety.
4. For each record:
   - We read the WAV file and perform harmonic-aware peak detection.
   - Upload the WAV file to `audio/ESC-50/<peak_freq>/`.
   - Append metadata with ESC-50-specific fields.
5. **Deduplication:** We skip records whose `source_id` (ESC-50 filename) already exists.
6. **Cleanup:** Lastly we delete the temporary dataset folder after ingestion.

Warm Path (near-real-time, user-interactive)

1. Here we record 5 seconds of microphone audio.
2. Perform harmonic-aware dominant frequency detection.
3. Generate a Cymatics **preview** video displayed to the user (looping playback), this video is **not** stored at this stage.
4. Then we prompt the user to approve or reject the recording.
5. If approved:
   - Uploads the audio WAV file to `audio/warm-path/<peak_freq>/`.
   - Appends a basic metadata row to CSV + Parquet.
6. If rejected:
   - Discards the observation entirely like never existed.

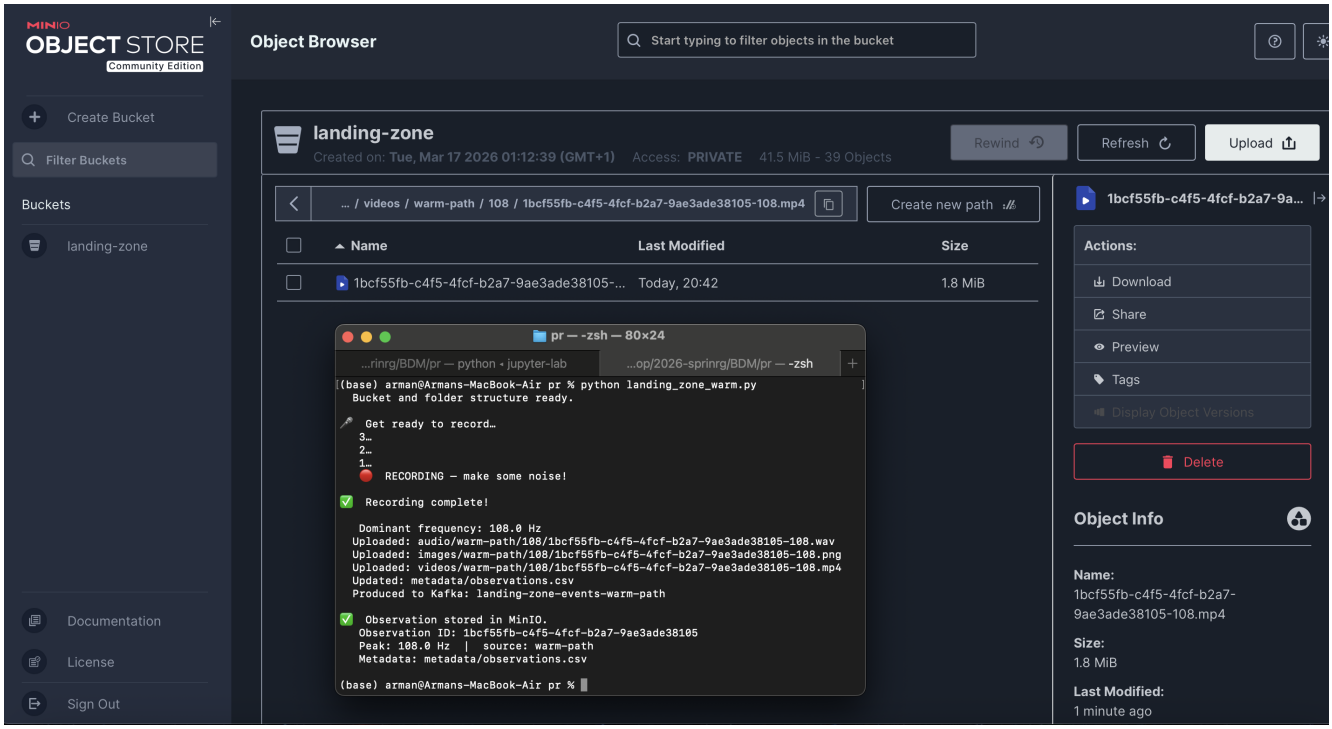

Figure 1: Warm Path Execution Example

Hot Path (real-time, producer/consumer with Kafka)

Producer (live view + raw upload)

- Displays a live cymatics visualization.
- Every **5 seconds**:
  - Uploads raw audio to `landing-zone/audio/hot-path/raw/`
  - Sends a metadata-only Kafka message(including audio path, bucket, timestamp, sample rate, device).
  - Flushes Kafka producer to ensure all messages recorded

Kafka message example

```
{
  "audio_path": "audio/hot-path/raw/abc123.wav",
  "bucket": "landing-zone",
  "timestamp": "2026-03-17T10:00:00Z",
  "sample_rate": 44100,
  "duration": 5
}
```

Consumer (move raw to structured)

- Here we Consume Kafka messages with audio path.
- Skip already processed data using CSV + memory tracking.
- Use manual Kafka offset commits.
- We download raw audio from MinIO.
- Perform harmonic-aware peak detection.
- Move audio from ‘raw/‘ to ‘audio/hot-path/<peak_freq>/<uuid>-<peak_freq>.wav‘, and deletes the raw copy.
- Then append a basic metadata row to CSV + Parquet.

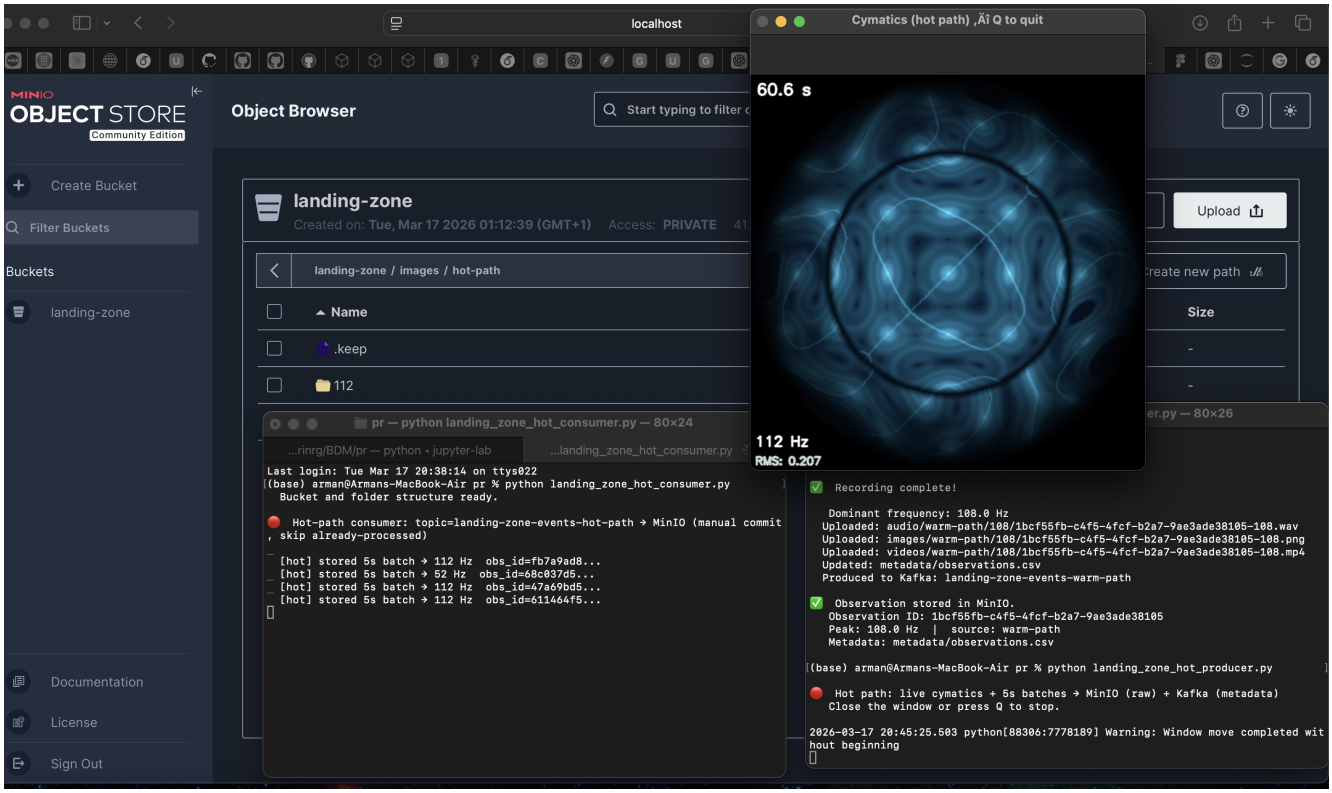

Figure 2: Hot Path Execution Example

Capabilities The Delta Lake table enables:

- ACID transactions on metadata
- Time-travel queries (version history)
- Schema evolution
- Efficient analytical queries via Parquet-based storage

3.2 Orchestration

Apache Airflow (automated scheduling)

A DAG file (`landing_zone/cold_path/dag_cold_freesound.py`) automates the Freesound cold-path ingestion, we were not able to add other paths(warm/hot) for automatic ingestion as there would be no microphone on airflow and neither any sound to record and ESC-50 is a pre-existing dataset that is never updated, only Freesound API is updated regularly and suits an automatic ingestion every week, on every run after ingestion, we decided to store the ingestion data to our bucket(MinIO) so that on next run we can read that date(if available) and filter records so that only the audios that were added **after** previous ingestion date, to only ingest recent records and spend less time for duplicate checking:

- **Schedule:** `@weekly` (every 7 days)
- **Batch size:** up to 250 newly added sounds per run
- **Incremental:** reads the last-ingestion checkpoint from MinIO and fetches only newly created sounds
- **Retries:** 3 retries with exponential backoff (5 min $\rightarrow$ 30 min max)
- **Execution timeout:** 2 hours
- **Logging:** DAG run ID, execution date, batch size, MinIO endpoint, elapsed time

For now Airflow runs as three Docker containers (init, webserver, scheduler) backed by PostgreSQL(for storing it's internal state), all defined in `docker-compose.yml`.

Manual Orchestrator (`orchestrate.py`)

Additionally, we provide a Python CLI for running any pipeline flow interactively, it displays all possible flows of this pipeline for user to choose, by choosing hot-path flow a producer and a consumer is launched in parallel, when user exits the producer, then consumer is automatically terminated and we return to first menu of this orchestrator, while processing any of the flows it also monitors and displays live metrics(CPU, RAM, disk) and updates them every 2 seconds to monitor lively, we are able to run SonarQube static analysis (requires `sonar-scanner` on the host path and SonarQube reachable at `http://localhost:9090`) and it is needed to provide a token for running the analysis.

3.3 Technologies Used

- **Object Storage — MinIO (S3-compatible)** was selected for its lightweight, container-friendly deployment and natural integration with docker.
- **Streaming — Apache Kafka + Zookeeper:** chosen for reliable real-time event streaming, allowing communication between producers and consumers.
- **Orchestration — Apache Airflow (DAGs) + manual CLI:** automated scheduling for batch ingestion and a manual CLI for interactive execution and real-time uses.
- **Metadata Formats — CSV, Parquet, Delta Lake:** CSV for simple uses, Parquet enables efficient columnar storing, and Delta Lake adds ACID transactions, time travel, ...
- **Audio Processing — NumPy, SciPy, `ffmpeg`:** Provides efficient numerical computation and robust audio decoding and signal processing capabilities.
- **Visualization — OpenCV:** Used for efficient generation of cymatics images and videos from computed wave interference patterns.
- **Static analysis - SonarQube + SonarScanner:** used for code-quality metrics and duplication tracking via local scans triggered by our orchestrator.

4 Architecture Design

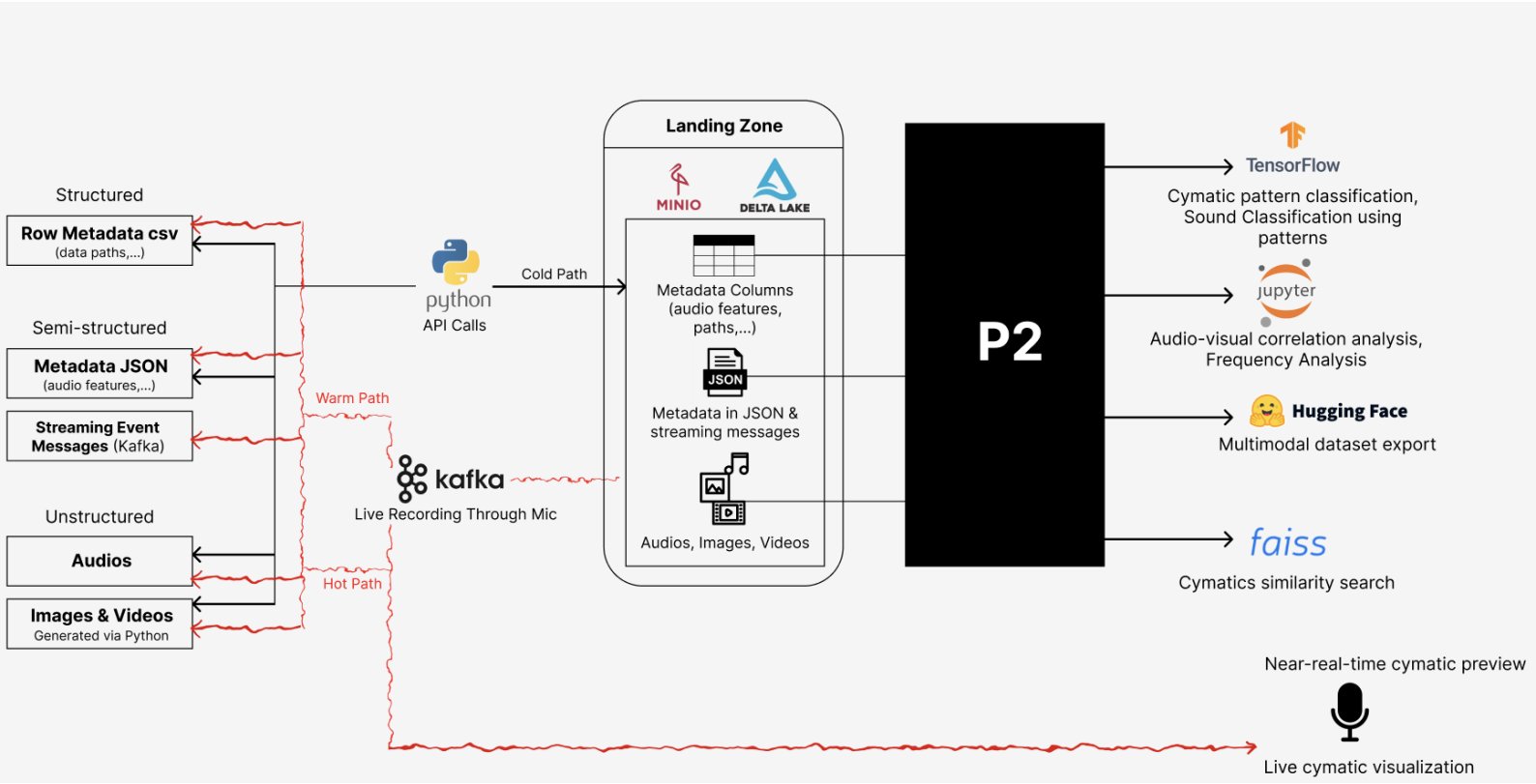

Figure 3: Architecture Design